

General Instructions

- Use Python language.
- Unless explicitly allowed, do not use built-in functions that directly perform the required image processing operation.
- Built-in functions for basic array operations, FFT/IFFT, image loading, and visualization may be used.
- Clearly state all parameter values used in the experiments.
- All input images and intermediate computations should be represented using floating-point values when necessary.
- Include figures, quantitative results, and a brief interpretation of each experiment.
- Your report should explain why the observed result occurs, not only show the output.

Point Processing and Histogram-Based Image Enhancement

You are given a color RGB image.

1. RGB to YUV Conversion

Convert the given RGB image into the YUV color space without using a built-in color conversion function.

Use

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.14713 & -0.28886 & 0.436 \\ 0.615 & -0.51499 & -0.10001 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}.$$

Display:

- Original RGB image
- Y component
- U component
- V component

Briefly explain the meaning of the Y , U , and V components. (Perform all computations using floating-point values. Do not clip or cast the U and V components to unsigned 8-bit integers during the intermediate computation.)

2. Histogram Computation

Using the luminance image Y , compute its histogram without using a built-in histogram function.

For an 8-bit image,

$$r_k \in \{0, 1, \dots, 255\}.$$

Compute

$$h(r_k) = n_k,$$

where n_k is the number of pixels having intensity r_k .

Display:

- Input luminance image
- Histogram
- Normalized histogram

$$p(r_k) = \frac{n_k}{MN}.$$

3. Histogram Equalization

Perform histogram equalization on the Y component without using a built-in histogram equalization function.

First compute the cumulative distribution function:

$$CDF(r_k) = \sum_{j=0}^k p(r_j).$$

Then obtain the transformed intensity:

$$s_k = (L - 1)CDF(r_k),$$

where $L = 256$.

Display:

- Original Y image
- Original histogram
- Equalized Y image
- Equalized histogram

Then combine the equalized Y component with the original U and V components and convert the image back to RGB.

Compare the original and enhanced RGB images.

4. Contrast Stretching

Implement contrast stretching without using a built-in contrast enhancement function.

Use a piecewise-linear transformation such as

$$s = \begin{cases} 0, & r < r_{\min} \\ \frac{r - r_{\min}}{r_{\max} - r_{\min}}(L - 1), & r_{\min} \leq r \leq r_{\max} \\ L - 1, & r > r_{\max}. \end{cases}$$

Select appropriate values of $r_{\min}$ and $r_{\max}$ using the 2nd and 98th percentiles of the luminance histogram.

Display:

- Original image
- Contrast-stretched image
- Histograms before and after stretching

Discuss how contrast stretching differs from histogram equalization.

5. Gamma Correction

Implement gamma correction using the power-law transformation.

First normalize the input intensity r to the range $[0, 1]$:

$$r_n = \frac{r}{255}.$$

Then apply

$$s_n = r_n^\gamma,$$

and convert the result back to the 8-bit intensity range:

$$s = 255 s_n = 255 \left(\frac{r}{255} \right)^\gamma.$$

Assume $c = 1$.

Test at least three values of γ , for example,

$$\gamma = 0.5, 1.0, 2.0.$$

Display the resulting images and explain:

- why $\gamma < 1$ brightens an image,
- why $\gamma = 1$ leaves the image unchanged,
- why $\gamma > 1$ darkens an image.

Also plot the corresponding input-output transformation curves.

6. Adaptive Histogram Equalization

Divide the luminance image into local regions (tiles) and perform histogram equalization independently in each region.

Do **not** use a built-in AHE function.

Display:

- Original image
- Global histogram equalization result
- AHE result

Discuss why AHE can improve local contrast better than global histogram equalization.

Also explain why AHE may amplify noise.

7. CLAHE

Implement Contrast Limited Adaptive Histogram Equalization (CLAHE) without using a built-in CLAHE function.

Your implementation should include:

1. Divide the image into tiles.
2. Compute the histogram for each tile.
3. Apply a predefined clipping limit.
4. Redistribute the clipped histogram counts.
5. Compute the local CDF.
6. Transform pixel intensities using the local CDF.
7. Interpolate between neighboring tiles to reduce blocking artifacts.

Test at least two different clipping limits.

Display:

- Original image
- AHE result
- CLAHE result
- Histograms of the results

Discuss:

- why CLAHE reduces noise amplification compared with AHE,
- how the clipping limit affects local contrast,
- what happens when the clipping limit is too small or too large.

8. Bit-Plane Slicing

Decompose the 8-bit luminance image into eight bit planes:

$$Y(x, y) = \sum_{k=0}^7 b_k(x, y) 2^k.$$

Display all eight bit planes.

Discuss:

- which bit planes contain most of the visual structure,
- which bit planes mainly contain fine details or noise.

Reconstruct the image using only the four most significant bit planes and compare it with the original.

9. Comparison and Discussion

Compare the following methods:

- Contrast stretching
- Histogram equalization
- AHE
- CLAHE
- Gamma correction

For each method, discuss:

- Global vs. local processing
- Contrast improvement
- Noise sensitivity
- Computational complexity
- Suitable applications

General Instructions

- Use Python language.
- Unless explicitly allowed, do not use built-in functions that directly perform the required image processing operation.
- Built-in functions for basic array operations, FFT/IFFT, image loading, and visualization may be used.
- Clearly state all parameter values used in the experiments.
- All input images and intermediate computations should be represented using floating-point values when necessary.
- Include figures, quantitative results, and a brief interpretation of each experiment.
- Your report should explain why the observed result occurs, not only show the output.

Spatial- and Frequency-Domain Filtering

You are given an original grayscale image $f(x, y)$.

For each filtering task, perform filtering in both the spatial domain and the frequency domain, and compare the results.

To make the spatial-domain and frequency-domain results comparable, carefully handle zero-padding and kernel alignment. Explain your implementation.

For visualization of Fourier transforms, use the centered log-magnitude spectrum:

$$\log(1 + |\text{fftshift}(F(u, v))|).$$

1. Blur Filtering

Use the following blur filter:

$$h_b(x, y) = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

Treat $h_b(x, y)$ as the impulse response of the blur system.

First, display the following:

1. Original input image

$$f(x, y)$$

2. Fourier transform of the input image

$$F(u, v) = \mathcal{F}\{f(x, y)\}$$

Display its log-magnitude spectrum.

3. Impulse response of the blur filter

$$h_b(x, y)$$

4. Fourier transform of the impulse response

$$H_b(u, v) = \mathcal{F}\{h_b(x, y)\}$$

Display its log-magnitude spectrum.

Next, perform filtering in the spatial domain:

$$g_s(x, y) = h_b(x, y) * f(x, y).$$

Then perform the same filtering in the frequency domain:

$$G_f(u, v) = H_b(u, v)F(u, v),$$

Implementation note:

When implementing frequency-domain filtering, use appropriate zero-padding so that the result corresponds to linear convolution rather than circular convolution. Use the same boundary condition for both spatial- and frequency-domain filtering.

and reconstruct the output image using

$$g_f(x, y) = \mathcal{F}^{-1}\{G_f(u, v)\}.$$

Also display the Fourier magnitude spectrum of the filtered output:

$$\log(1 + |G_f(u, v)|).$$

Compare the spatial-domain and frequency-domain results using:

MSE, PSNR, SSIM.

Discuss whether the two filtering results are identical or nearly identical.

2. Sharpening Filtering

Use the following sharpening filter:

$$h_s(x, y) = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}.$$

Treat $h_s(x, y)$ as the impulse response of the sharpening system.

Display:

1. Original input image

$$f(x, y)$$

2. Fourier transform of the input image

$$F(u, v) = \mathcal{F}\{f(x, y)\}$$

3. Impulse response

$$h_s(x, y)$$

4. Fourier transform of the impulse response

$$H_s(u, v) = \mathcal{F}\{h_s(x, y)\}$$

5. Spatial-domain filtering result

$$g_s(x, y) = h_s(x, y) * f(x, y)$$

6. Frequency-domain filtering result

$$G_f(u, v) = H_s(u, v)F(u, v)$$

and

$$g_f(x, y) = \mathcal{F}^{-1}\{G_f(u, v)\}.$$

7. Fourier magnitude spectrum of the filtered output

$$\log(1 + |G_f(u, v)|).$$

Compare the spatial- and frequency-domain results using:

MSE, PSNR, SSIM.

3. Frequency-Domain Analysis

For both the blur and sharpening filters, compare the following three spectra:

$$\begin{aligned} &|F(u, v)|, \\ &|H(u, v)|, \end{aligned}$$

and

$$|G(u, v)| = |H(u, v)F(u, v)|.$$

Explain how the frequency response $H(u, v)$ modifies the frequency components of the input image.

Discuss the following:

- Why does the blur filter behave as a low-pass filter?
- Which frequency components are attenuated by the blur filter?
- Why does the sharpening filter emphasize high-frequency components?
- How are edges and fine details related to high-frequency components?

4. Verification of the Impulse Response

Create a 2-D discrete impulse image:

$$\delta(x, y) = \begin{cases} 1, & x = 0, y = 0 \\ 0, & \text{otherwise.} \end{cases}$$

Apply both the blur and sharpening filters to the impulse image.

Verify that

$$h(x, y) * \delta(x, y) = h(x, y).$$

For each filter, display:

- impulse input $\delta(x, y)$
- Fourier transform of the impulse
- output of the filtering operation
- impulse response $h(x, y)$
- Fourier transform $H(u, v)$ of the impulse response

Explain why $h(x, y)$ is called the **impulse response** of the system.

5. Gaussian-based Image Sharpening

Apply image sharpening using a Gaussian low-pass filter.

First, generate a Gaussian kernel $h_G(x, y)$ with a selected standard deviation σ , and obtain the blurred image

$$f_L(x, y) = h_G(x, y) * f(x, y).$$

Then obtain the high-frequency component by

$$f_H(x, y) = f(x, y) - f_L(x, y).$$

Construct the sharpened image using

$$g(x, y) = f(x, y) + kf_H(x, y),$$

where $k > 0$ controls the sharpening strength.

Equivalently,

$$g(x, y) = (1 + k)f(x, y) - kf_L(x, y).$$

Perform the experiment using at least two different values of σ and k .

Display:

- Original image $f(x, y)$
- Gaussian kernel $h_G(x, y)$
- Gaussian-blurred image $f_L(x, y)$
- High-frequency component $f_H(x, y)$
- Sharpened image $g(x, y)$

Also display the Fourier magnitude spectra of the original, blurred, high-frequency, and sharpened images.

Discuss:

- Why does subtracting the Gaussian-blurred image extract high-frequency information?
- How does σ affect the extracted high-frequency components?
- How does k affect the sharpening strength?

- What happens when k is too large?
- Compare this method with the fixed sharpening kernel used in the previous experiment.

6. Discussion

Summarize your results and discuss the following relationship:

$$g(x, y) = h(x, y) * f(x, y)$$

and

$$G(u, v) = H(u, v)F(u, v)$$

Explain how convolution in the spatial domain corresponds to multiplication in the frequency domain.

Also discuss why the spatial-domain and frequency-domain results may show small differences in practical implementation, considering:

- zero-padding,
- circular convolution,
- boundary handling,
- numerical precision.

General Instructions

- Use Python language.
- Unless explicitly allowed, do not use built-in functions that directly perform the required image processing operation.
- Built-in functions for basic array operations, FFT/IFFT, image loading, and visualization may be used.
- Clearly state all parameter values used in the experiments.
- All input images and intermediate computations should be represented using floating-point values when necessary.
- Include figures, quantitative results, and a brief interpretation of each experiment.
- Your report should explain why the observed result occurs, not only show the output.

Wiener Filter

You are given an original grayscale image $f(x, y)$ attached.

The degradation model is

$$g(x, y) = h(x, y) * f(x, y) + n(x, y),$$

where $h(x, y)$ is the point spread function (PSF), $*$ denotes convolution, and $n(x, y)$ is additive noise.

The PSF is given by

$$h(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right),$$

with

$$\sigma = 2.5.$$

Use a 15×15 discrete PSF kernel and normalize it such that

$$\sum_y h(x, y) = 1$$

1. Generate a blurred image

Generate a blurred image using the given PSF:

$$g_b(x, y) = h(x, y) * f(x, y).$$

Show the original image, the PSF, and the blurred image.

Add zero-mean Gaussian noise to the blurred image:

$$g(x, y) = g_b(x, y) + n(x, y).$$

The RMS value of the noise is given as

$$\text{RMS}_{\text{noise}} = 0.03.$$

Generate the noise so that its actual RMS value is approximately equal to the specified value.

Show the blurred image and the blurred + noisy image.

2. Add noise

Add zero-mean Gaussian noise to the blurred image:

$$g(x, y) = g_b(x, y) + n(x, y).$$

The RMS value of the noise is given as

$$\text{RMS}_{\text{noise}} = 0.03.$$

Generate the noise so that its actual RMS value is approximately equal to the specified value.

Show the blurred image and the blurred + noisy image.

3. Restore the image using a Wiener filter

Design a Wiener filter for the degraded image in Step 2:

$$\hat{F}(u, v) = \frac{H^*(u, v)}{|H(u, v)|^2 + K} G(u, v),$$

where $H(u, v)$ is the Fourier transform of the PSF.

Restore the image using the inverse Fourier transform.

Do not use a built-in Wiener filtering function.

4. Investigate the effect of K

Perform Wiener filtering using several different values of K , for example,

$$K = 10^{-6}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}.$$

Show the restored images for different values of K , and discuss:

- What happens when K is too small?
- What happens when K is too large?
- How does K affect noise amplification and image sharpness?
- Which value of K gives the best restoration result, and why?

Optionally, compare the results using PSNR, MSE and SSIM.

For each value of K :

- Show the restored image.
- Compute the **MSE**, **PSNR**, and **SSIM** between the restored image and the original image.
- Summarize the results in a table.

K	MSE	PSNR (dB)	SSIM

